SIADS 696: Team 14 - Philadelphia Bike Share

Xiaofan Tang, Ian McCourt, Hal Newton

Introduction

Indego, Philadelphia's public bike-share system, recorded more than 3.3 million trips between January 2024 and June 2026. Every hour some docks run empty while others fill, and rebalancing crews can act ahead of time only if they know where demand is heading. We therefore ask: given what is known at the start of an hour, how many trips will start at each station over the next few hours? For future work, we also ask whether forecasts that work in ordinary weeks still work when travel conditions change abruptly. We study two such periods as case studies, not modeling targets. The first is the 20% SEPTA Bus and Metro service cut (August 24 to September 13, 2025), followed by a 21.5% fare increase on September 14 (City of Philadelphia, 2025). The second is the five FIFA World Cup group-stage matches played in Philadelphia between June 14 and June 27, 2026.

This matters because disruptions empty stations fastest and forecast errors cost the most. For the World Cup, the City asked visitors to walk, bike or ride transit and added a temporary 100-bike Indego hub at the Fan Festival (City of Philadelphia, Office of Special Events, 2026). A model that is accurate on average but fails at those moments would mislead the people who rely on it most.

Related work

Indego demand prediction (Chen, 2019). Chen aggregated April–June 2019 Indego trips to station-hours and fit linear regressions with station and hour effects, weather, census attributes and 1-hour to 7-day lags, reporting weekly test MAE of roughly 0.7–0.9 trips. We use the same city and a similar panel, but 2024–2026 data, count and tree models, forward-chaining validation, prior-hour rather than same-hour weather, event-period error analysis and a clustering component.

Cluster-level prediction (Li et al., 2015). Li and colleagues clustered New York and Washington, D.C. stations with a bipartite clustering algorithm, predicted city-wide rentals with gradient-boosted regression trees and allocated them across clusters, with the largest gains in anomalous periods. We also pair clustering with boosted trees, but predict each station directly, treat clusters as optional features whose value we measure, and build clusters from inferred trip purpose rather than geography and transitions.

Biking usage clustering and prediction (Froehlich et al., 2009). Using 13 weeks of Barcelona station data, Froehlich and colleagues clustered stations by temporal usage and compared four predictors of near-term bike availability (last value, historic mean, historic time-of-day mean and a Bayesian network), finding recent observations more useful than long-run averages. Our historical baselines play the role of their simple predictors; we forecast departures rather than availability, over a far longer record, and evaluate errors during a transit cut and a World Cup.

Data Sources

The table in Appendix D1 summarizes every dataset. All sources are public. The DisPatch70 Hugging Face repository (hulnegy, n.d.) has no dataset card, so column meanings come from the files themselves.

Indego trips are the core dataset: one row per completed rental with local start and end times. Indego already removes staff and test trips and trips under one minute, caps durations at 24 hours, and routes remote staff check-ins to a “Virtual Station” (Indego, n.d.). The median trip lasts ten minutes (99th percentile 83), the median one-way distance is 1.56 km and 6.2% are round trips. In Q3 2024–Q2 2026, 65.3% of trips used electric bikes and members dominated (Indego30 52.3%, Indego365 37.1%). Trips record completed rentals only; requests that failed at an empty dock are invisible, so departures are a censored measure of demand.

DisPatch70 is a multi-city bike-share operations and context collection (716,426,797 rows across all cities); we used only the Philadelphia tables. Weather joins to every hour, but dock status does not (Figure 1): there are no status files for January–March or May–July 2024, published months from late 2024 cover only about 24–48% of station-hours, and October 2025 covers 4.9%. We therefore treat status as optional. Dispatch rows name stations rather than giving IDs; 321 of 326 names matched after normalization, 1 by manual alias, 3 by fuzzy matching and 1 (Aramingo & Cumberland) not at all.

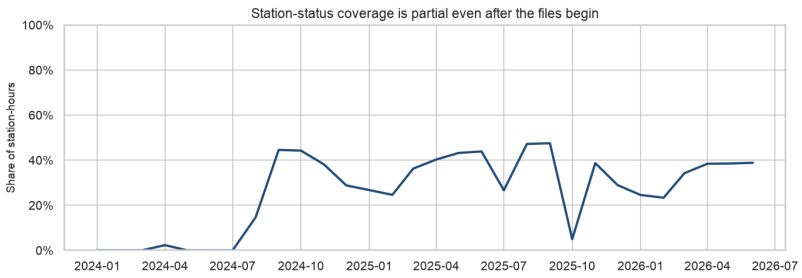


Figure D1: Share of station-hours with a DisPatch70 station-status snapshot, by month. Weather coverage is 100% in every month.

Feature Engineering

Schema Checks and Cleaning.

We concatenated the ten quarterly files after normalizing column names and stopped on any file missing one of the 13 trip fields the pipeline needs, guarding against silent layout changes. Times were parsed with mixed-format handling and station IDs taken from the trailing digits of the station field. Because Indego pre-cleans its files, only 134 rows were removed (Table 2): 133 Virtual Station starts (a staff bucket, not a place riders can reach) and one start outside a Philadelphia bounding box. We kept 51,217 trips (1.5%) ending at the Virtual Station or nowhere; they count as departures but not as arrivals. Trip distance is the great-circle distance between endpoints, left missing when the end lies outside the bounding box.

Table D2. Trip cleaning steps (cleaning_report.csv)

Step	Rule	Rows
Rows read	All 2024 Q1–2026 Q2 files	3,316,798
Bad start time	Unparseable start_time	0
Missing start station	No numeric start-station ID	0
Duplicate trip	Same trip_id and start time	0
Duration out of bounds	Outside 1–1,440 minutes (Indego’s published limits)	0

Virtual start	Start station 3000	133
Start outside Philadelphia	Start outside lat 39.80–40.15, lon −75.35 to −74.90	1
No real end station (kept)	End at station 3000 or missing; counted as departure only	51,217
Outside study years	Start year not 2024–2026	0
Rows kept		3,316,664

Station Service Window

Filling every station × hour with zeros would create false zeros before a station opened or after it closed. Each station instead enters at its published go-live date (or first trip, if earlier) and stays until the window end if Active or its last trip if Inactive, giving 339 stations and 5,950,045 in-service station-hours in which hours with no trips are true zeros. Departures are counted at the start station and hour, arrivals at the end station and hour, and net flow is their difference. Panel departures correspond exactly to the 3,316,664 kept trips.

Joining context without inventing values. We averaged weather from the 30-minute snapshots within each hour (the typical_trips profile was summed; holiday and day-type fields took the first valid value). Weather-event intervals were expanded onto every overlapped hour as eight flags. Sports rows were given coordinates for seven venues; each station-hour kept the nearest overlapping final game, with near_sports meaning within 1.5 km. Dock snapshots were converted from UTC to local time and averaged per station-hour, with zero_bikes and full_station taken from the hourly minimum; hours without a snapshot stay missing, because no reading is not the same as “no bikes”. Rebalancing moves add bikes at the destination and remove them at the origin, with zeros filled only in years whose file was loaded.

Supervised Learning

We compared Poisson regression, XGBoost and a neural network to predict departures at Philadelphia bike stations. In summary, XGBoost had the lowest test MAE of 1.4138 departures per four-hour station window, about 5.3% lower than the strongest historical baseline. Recent demand was the most useful information for prediction, although the model still struggled with zero-departure windows and sudden increases in demand.

Prediction Data and Methods

At forecast time T, we use observations from completed hours through T − 1 to predict departures over the next four hours, [T, T + 4 hours). For example, a forecast at 15:00 covers departures from 15:00 to 19:00, excluding 19:00 (all times are in Philadelphia local time). We selected the four-hour window after comparing different forecast lengths through validation, which will be discussed later. We make a new forecast every hour, so neighboring four-hour windows overlap. Note that the target only counts completed trips, meaning it does not include people who wanted a bike but could not get one.

The original hourly dataset contains 4,640,783 training rows from January 2024 to December 2025, 640,874 validation rows from January to March 2026, and 668,043 test rows from April to June 2026. After keeping only

complete four-hour targets, the test set contains 667,101 windows across 314 stations. Our initial checks found no duplicate rows or invalid numeric values in the loaded columns. However, during feature construction, we found 775 negative station ages. We treated these ages as missing but kept their departure counts, since incorrect station information did not mean the trips themselves were invalid. We also kept genuine zeros and large counts.

We started with 22 main features, including calendar information, recent departures and arrivals, net flow, matching previous-day and weekly totals, the four-week average, the number of available weeks, five weather measurements from the previous hour, and station ID, coordinates and age. We later tested additional features related to behavior, sports, bike status and station context. The final feature set includes 5 extra status inputs: bike and dock readings, indicators for empty and full stations, and whether a status reading was available. Feature selection process and reasoning will be discussed in later sections. Station groups identified in the unsupervised analysis were used separately to examine whether prediction errors differed across stations with different trip patterns.

Before training, we apply log(1 + x) to historical counts and a signed log transformation to net flow. We use sine and cosine terms for calendar cycles and one-hot encoding for categorical features, ignoring categories not seen during training. Imputation and scaling are fitted separately for each training run. Poisson regression and the neural network use median filling and standardized numeric inputs, while the selected XGBoost model handles missing numeric values directly. We also use dense inputs so XGBoost can distinguish missing values from actual zeros. Negative predictions are set to zero, which mainly affects the neural network.

Table S1: Model families and initial tuning grid

Model and reason for using it	Values tested	Fixed settings
Poisson regression: Simple count model with log link and L2 penalty	Alpha 0.01, 0.1, 1.0	Maximum 300 iterations Tolerance 0.00001
XGBoost: Boosted trees for nonlinear thresholds and interactions	Depth 3, 5 Trees 100, 200	Poisson objective Learning rate 0.05 Row/column sampling 0.9 L2 lambda 5
Neural network: A hidden layer learns nonlinear patterns in another way	Width 32, 64 Alpha 0.01, 0.1	Adam, maximum 300 iterations Batch 1,024 Learning rate 0.001 No early stopping

Evaluation Setup

We used MAE as the main measure for comparing models, since it shows how far predictions are from actual departures on average. We also report RMSE, which gives more weight to larger errors, and bias, which measures whether predictions tend to be too high or too low. Percentage error was not suitable because many windows had zero departures. For comparison, we used three historical baselines: the station-weekday-hour average, departures during the same window in the previous week, and the average of up to four matching weekly windows. When weekly data were unavailable, we used the historical average instead, with city-level or overall averages as fallbacks for unseen stations.

We used forward validation to see how well the models predicted later periods using only earlier training data. The five evaluation periods covered each quarter of 2025, followed by January to March 2026. For each period,

training targets ended before evaluation began, so departures from the evaluation period were not used for training. The initial comparisons used up to 50,000 training rows and the same 20,000 forecast starts per fold. Historical averages used all eligible earlier data. Model seeds were set to 42, and sampling seeds were 42 plus the fold number.

We first compared the models using one-hour forecasts, where XGBoost had the lowest mean validation MAE of 0.478. This gave us a starting point, but we did not choose the final model yet. We then compared longer forecast windows before using the best-performing setup to test missing-value methods, training sizes and feature combinations.

Sensitivity Analysis and Training Size

Forecast-window sensitivity

To choose the forecast window, we compared one- to four-hour totals using all three model grids, the 22 main features and median filling. We used the same forecast starts for each duration, making sure every start had a complete four-hour target. For each duration, we selected the model and settings with the lowest mean validation MAE. We then divided that model's MAE by the best baseline MAE within each fold and averaged the ratios. This allowed us to compare improvement over historical predictions across different window lengths. Since the strongest baseline could differ between folds, the result is not the same as dividing the overall average model MAE by the overall average baseline MAE.

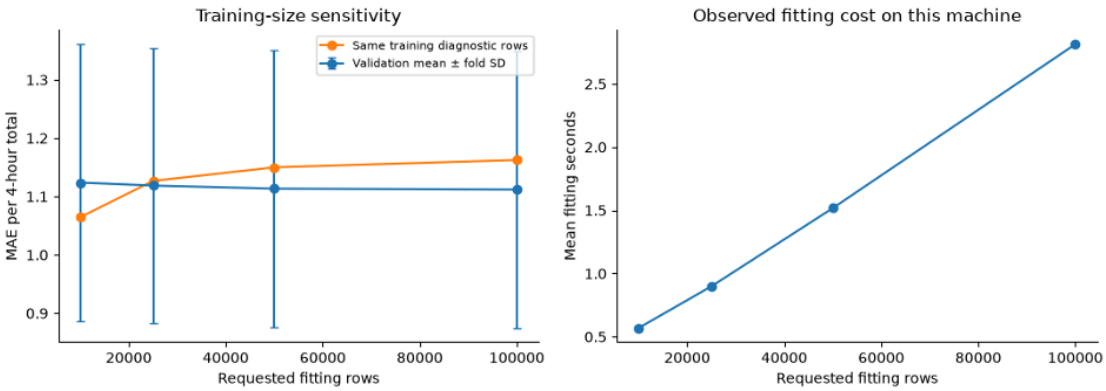
Table S2: Sensitivity to the forecast window

Hours	Mean MAE	MAE per hour	Relative MAE ± SD
1	0.478	0.478	1.000 ± 0.043
2	0.728	0.364	0.967 ± 0.042
3	0.933	0.311	0.948 ± 0.041
4	1.115	0.279	0.935 ± 0.042

Missing-value sensitivity

The four-hour window had the lowest mean relative error of 0.935, with XGBoost performing best at that duration. We therefore kept this combination for the next experiments. However, dividing four-hour MAE by four does not tell us how accurate each individual hour is, since combining departures can hide timing errors. We next compared three ways of handling missing values using the selected XGBoost settings. Native handling, median filling and zero filling gave mean validation MAEs of 1.1153, 1.1155 and 1.1159. We kept native handling because it performed slightly better, although the differences were very small. Median and zero filling also included missing-value indicators, so the comparison involved more than just how missing values were replaced.

Figure S3: Training-size sensitivity for preliminary XGBoost. Error bars show fold SD. Training error is measured on the same rows at every sample size. Fitting times apply to the machine used for this run (in this case, local).



Training size

With the forecast window and missing-value method selected, we looked at whether using fewer training rows could save time without losing much accuracy. We kept the XGBoost settings and 22 main features the same. Using 100,000 rows gave the lowest mean validation MAE of 1.1119. With 25,000 rows, MAE was 1.1188, only about 0.6% higher, while average fitting time dropped from 3.03 to 1.00 seconds. We chose 25,000 as the smallest sample within 1% of the best result. The 10,000- and 50,000-row samples gave MAEs of 1.1238 and 1.1135. This gave us a reasonable balance between accuracy and fitting time.

In the training-size experiment, within each fold, larger samples kept all the rows from smaller samples and added more, so differences in performance were less affected by which rows were selected. Note that later experiments used a different shared sample of 25,000 rows, which explains why the main-feature MAE was 1.1191 instead of 1.1188.

The training-size experiment also compared training and validation errors. Training errors were measured on a fixed sample from the earlier training period, while validation errors were measured on later dates. A larger validation error could suggest overfitting, but it could also reflect changes in demand between these periods, so the difference alone cannot tell us how much overfitting occurred. Although we tested different tree depths and numbers of trees during model tuning, we did not dive deep into their individual effects on MAE. We therefore have stronger evidence for the effects of training size than for sensitivity to specific XGBoost settings.

Feature Selection and Final Model Comparison

After choosing the training size, we tested whether adding other information could improve XGBoost's predictions. We kept the preliminary model settings the same and compared the 22 main features with five alternatives: removing weather or adding behavior, context, sports or bike status. Bike status gave the largest improvement, reducing mean validation MAE from 1.1191 to 1.1085. Context, behavior and sports gave smaller improvements, with MAEs of 1.1171, 1.1179 and 1.1180. Removing weather increased MAE to 1.1368. Based on these results, we selected the 27-feature set containing the main features plus bike status. However, since we tested each addition separately, we did not search for the best combination of all optional features.

Since the initial hyperparameter tuning took place before we finalized the forecast window, missing-value method, training size and feature set, we tuned all three models again using the selected four-hour window, 25,000 training rows and 27 features. XGBoost kept native missing-value handling, while Poisson regression and the neural network used median filling and scaling. This allowed us to compare the models under the same conditions. However, because we used XGBoost to choose the training size and features, the results do not necessarily represent the best performance each model could achieve with its own separately selected inputs.

Table S4: Final four-hour comparison across five forward folds

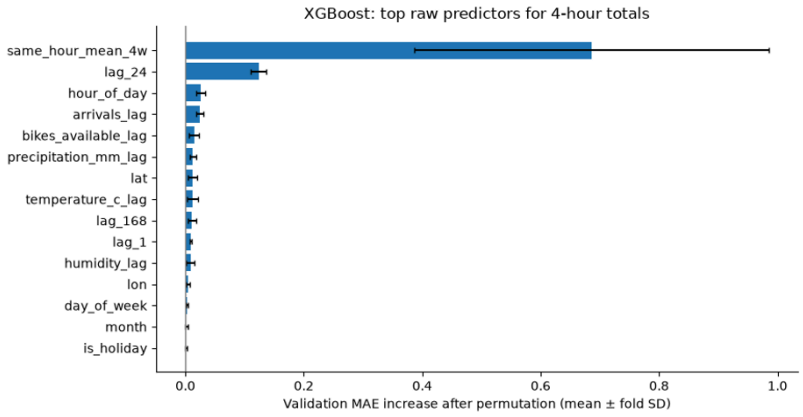
Model or baseline	MAE mean ± SD	RMSE mean ± SD
XGBoost	1.1085 ± 0.2309	1.8013 ± 0.3056
Poisson regression	1.1785 ± 0.2298	1.8830 ± 0.3787
Previous four weeks mean	1.1962 ± 0.2547	2.0257 ± 0.3279
Neural network	1.2250 ± 0.2359	1.9424 ± 0.3094
Station weekday hour mean	1.3891 ± 0.1318	2.2888 ± 0.2640
Same hour previous week	1.4027 ± 0.3242	2.4730 ± 0.4437

XGBoost still had the lowest validation MAE of 1.1085, using depth 5 and 200 trees, so we kept it as the final model. Poisson regression selected alpha 0.01, while the neural network used 32 hidden units and alpha 0.1. The network reached its 300-iteration limit in all five folds, suggesting it had not fully converged. Its performance therefore only reflects the training limit we used, and further training might change the results. Besides, since we used the same five validation folds to select the model and its settings, the final validation MAE may be somewhat optimistic. We therefore also evaluated the selected model on the later April-June 2026 test period.

Feature Importance and Ablation

After selecting XGBoost, we looked more closely at which features it relied on for predictions. We used permutation importance, which measures how much the error increases when one input is shuffled while the fitted model stays the same. Each raw input was shuffled before transformation, with five repeats on up to 5,000 validation rows per fold. The four-week average had the largest effect, increasing MAE by 0.6857, followed by the previous-day total at 0.1237. Shuffling related inputs together also showed that demand history was the most important group. However, this tells us which inputs the model uses, not what causes demand. Related features may contain similar information, and shuffling can create combinations that would not normally occur.

Figure S5: Raw inputs ranked by permutation importance for the selected XGBoost model. Bars show the mean increase in validation MAE, with fold SD as error bars. The chart shows the top 15 of 27 inputs.



We also used ablation to see how well the model could perform without certain feature groups. We grouped the 27 features into five categories: demand history, calendar, weather, bike status, and station identity and location. Unlike permutation importance, ablation removes a feature group and retrains the model using the remaining inputs. This allows the model to adjust and rely more on other features. We kept the same settings, folds and sampled rows for a fair comparison. Removing demand history had the biggest impact on performance. Removing bike status also made predictions worse across all five folds, suggesting that these features are still useful even with many missing readings. Together, the two methods help us understand which features the model relies on and how well it performs when certain information is no longer available.

Table S6: Ablation results after retraining the selected model

Removed feature group	Mean MAE	Paired MAE change ± SD	Folds worse
None	1.1085	0.0000	0
Demand history	1.3918	+0.2833 ± 0.0414	5
Prior weather	1.1272	+0.0187 ± 0.0143	4
Calendar	1.1263	+0.0178 ± 0.0145	5
Bike status	1.1191	+0.0106 ± 0.0080	5
Station identity and location	1.1101	+0.0016 ± 0.0024	4

Test Evaluation

After selecting the final model settings, we fitted all three models on the original pre-2026 training data, using 25,000 sampled rows without adding the January-March 2026 validation targets. We then evaluated them on 667,101 four-hour windows from April to June 2026.

Table S7: April-June four-hour test results

Model or baseline	MAE	RMSE	Bias
XGBoost	1.4138	2.6096	- 0.1024
Previous four weeks mean	1.4933	2.8001	- 0.2909
Poisson regression	1.5161	2.7477	+ 0.0709
Neural network	1.5438	2.7575	- 0.0516
Station weekday hour mean	1.6771	3.1888	- 0.8253
Same hour previous week	1.8317	3.4467	- 0.0953

XGBoost achieved the lowest test MAE of 1.4138, compared with 1.4933 for the four-week baseline, an improvement of about 5.3%. Its RMSE was 2.6096 and bias was -0.1024, while Poisson regression and the neural network had higher errors.

Error Patterns Coverage and Tradeoffs

Although XGBoost reduced test MAE by about 5.3%, we also wanted to estimate how much this improvement might vary across different samples of dates. We calculated the difference in absolute error between XGBoost and the baseline for each matching station and four-hour window, then averaged these differences. This gave a paired MAE difference of approximately -0.079 departures. Keeping the errors paired ensures both models are compared under the same conditions. To estimate uncertainty, we resampled seven-day blocks from the 91 test dates 1,000 times, preserving some dependence between nearby dates. We also kept all stations from each date together to account for shared daily conditions. Circular sampling allowed blocks reaching June 30 to continue from April 1. The resulting approximate 95% bootstrap confidence interval was [-0.110, -0.055]. This suggests the improvement is fairly consistent under the chosen resampling method.

We also compared errors across different categories of test windows to see where the model performed well and where it struggled. As shown in Appendix S1, these included windows with zero, positive or high demand, limited

historical data, and periods near the stadium with or without match overlap. We used the same four-week baseline throughout and defined high demand as at least 9 departures, the 95th percentile of the training targets. Some windows belonged to multiple categories, which also differed in size and average demand. For example, high-demand windows may naturally have larger errors because there are more departures to predict. We therefore cannot conclude from MAE alone that a particular condition caused the model to perform worse.

The model did not improve on the baseline when no departures occurred, with an MAE of 0.664 compared with 0.557 for the baseline. Errors were also much higher near the stadium during match-overlap windows than during other June windows. However, only 245 windows overlapped with matches, so we cannot conclude that the events caused the larger errors. The model also had a negative overall bias, meaning it tended to underestimate departures on average. However, bias alone does not show how accurate the predictions were, since overpredictions and underpredictions can cancel each other out.

Missing Status and Station-Group Analysis

Although bike-status features helped improve predictions, their availability remains a concern. Previous-hour readings were missing in 75.4% of the original training rows and 61.4% of the test rows. XGBoost can still use these rows without filling in the missing values, but it cannot recover information that was never recorded. The results show that status features were useful with the data available, but we cannot assume these readings will always be available when the model is used in practice.

We also used the station groups from the earlier unsupervised analysis to examine how prediction errors varied across stations with different trip patterns. The clustering analysis had assigned 229 stations to groups based on their trip history. Of the 314 stations in the test data, 225 had a group assignment and 89 did not, although all were included in the supervised evaluation. Among the 89 ungrouped stations (details in Appendix S2), 66 appeared in the training data and 23 were absent from it. Of the 66 seen during training, 65 had positive departures, while station 3004 had none. This shows that lacking a clustering group does not necessarily mean a station was unseen during training or had no departure history.

Comparing errors across these station groups helps us understand whether the model performs differently for stations with different trip patterns. XGBoost had lower MAE than the historical baseline in every group, although the errors varied. For example, as shown in Appendix S3, the short-hop core had an MAE of 1.7786, compared with 0.9983 for leisure and parks stations. Ungrouped stations had a lower MAE of 1.0304, but their average demand was also lower than in the short-hop core (1.7031 versus 4.1284 departures). These comparisons show how clustering can help us identify differences in forecasting performance, although we cannot tell whether those differences are related to trip patterns or other factors, such as demand levels.

Overall, the smaller training sample reduced fitting time, historical inputs were relatively easy to interpret, and bike status improved accuracy. The station-group comparisons also provided another way to examine where prediction errors were larger. However, the model still struggled with timing errors, sudden demand increases and missing optional data, which could limit its usefulness for station operations. The following cases look more closely at three individual station-days where the model made particularly large errors.

Three Failures Cases

To better understand where the model struggled, we selected one difficult station-day from each of three different contexts. These examples help illustrate possible weaknesses, but they do not tell us how often the errors occur or what caused them. The table in Appendix S4 reports the actual four-hour departures, with surrounding patterns shown in Appendix S5.

Case 1: Underprediction during an event window

At Broad and Pattison, the model predicted 9.725 departures, far below the actual 111. The station is 0.663 km from the stadium, and the forecast overlapped with a recorded match period. Historical demand was much lower, with 12 departures in the matching previous-day window, 2 in the previous-week window, and 4 in the preceding hour despite 22 arrivals. Adding sports features only slightly improved overall validation MAE, so they were not selected for the final model. However, this case suggests that combining event timing with station location may help predict similar surges. We would need to verify the trip records and event schedules before linking the surge to the match, then test any improvements on other event dates for future study.

Case 2: Overprediction when no departures occurred

At 30th Street Station East, the opposite happened. The model predicted 25.603 departures and the baseline predicted 28, but no departures actually occurred. The matching previous-day and previous-week totals were 13 and 26, with all four weekly lags available. The preceding hour also had an average of 11 available bikes, so we cannot assume the station was empty. This shows how the model can struggle with zero-departure periods, even when historical demand suggests some activity. Possible explanations include low demand, a temporary service interruption or a recording problem. We would first check the operating and trip records for that period, then test whether a separate model for zero versus positive departures or reliable interruption indicators could help.

Case 3: Limited history with unusual inflow

Lemon Hill West was absent from the pre-2026 training data and had only two weekly lags available. It recorded 304 departures, much higher than the model's prediction of 16.251 and the baseline's 13.5. Limited historical data may have contributed to this error, but it does not fully explain it. The preceding hour had already recorded 143 arrivals and a net inflow of 133 bikes, indicating unusual activity before the forecast. This suggests it may be worth testing whether information from similar stations and features that capture large inflows could improve predictions on later validation dates. Although the station experienced unusually high demand, the 15:00-19:00 forecast window did not overlap with the recorded evening kickoff, so we cannot describe this as a match-hour prediction failure.

Unsupervised Learning

Motivation

The station dataset appears basic, with four dimensions. Fortunately, the ride or trip data includes a few critical pieces, including lat/lon and the start and end times of a trip. Relying solely on lat/lon features would likely only produce geographic station segmentation.

For our project, we will use a two-phase clustering process. We will reveal the latent structure in the trip purposes each station serves. We then use these purpose-based clusters to characterize the stations. We will treat these clusters as segments rather than station types, characterizing each station by its proportional mix of purposes rather than only the most common purpose for rides at that station. The results might influence predictions in the supervised learning phase.

Data Source

To avoid burdening the narrative with excessive detail on data processing in this phase, refer to Appendix X for the schema and other background information on the datasets.

To prepare the ride data for this analysis, we dropped the virtual station with no coordinates (id = 3000), round trips of two min or less (false starts and fast redocks), and trips over four hours (assume lost or overnight rides). Although pass type was available, given the pricing structure and consumer arbitrage opportunities to minimize cost per ride, we cannot infer ride type from pass data. The most dominant pass type fits both commuters and tourists looking to minimize cost per ride.

Unsupervised Learning Methods

Clustering Approaches

To explore clustering, we selected both deterministic (k-means) and probabilistic (Gaussian Mixture Model) methods. We can use the trip dataset's datetime information to derive additional categorical and numerical fields for both approaches.

We start with trip data, also called ride data in this report, from the training period (January 2024-December 2025). We can infer each ride's purpose, even though the dataset lacks an explicit feature for it. Appendix U1 shows how we derive new features from existing ones. This purpose-centric analysis has two phases:

- Trip purpose: Each trip is described by duration, straight-line distance, straight-line speed, round-trip flag, start time of day (cyclical) and weekend flag. Pass type and bike type are held out
- Station types: Each station is described by its ride purpose mix: the proportion of trips starting or ending there that are characterized by a purpose from phase 1.

Trip Clusters

K-means Trip Clusters

By evaluating both the inertia and silhouette scores, we can make a judgment call on the most appropriate k-value. We must balance interpretability, separation effectiveness, and cluster diversity to support the supervised learning task. We settled on k=6 after balancing these trade-offs. Time dominates the definition of purpose, but that is intentional, as it captures both commuting and recreational purposes and their variations. Duration and distance are the next strongest pair, which fits how rides might naturally imply purpose. Refer to Appendix U1 for additional visualizations comparing each of the clusters for both methods.

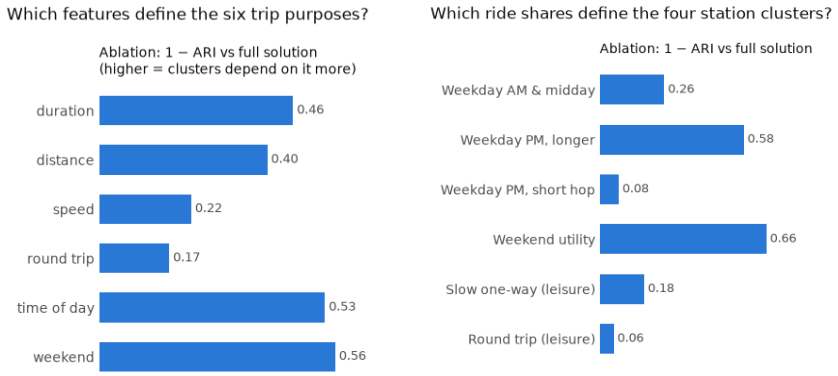


Figure U1. Features that define the six trip purpose clusters and four station clusters

GMM Trip Clusters

Given that each station has a proportional mix of purposes, a probabilistic approach like Gaussian Mixture Model (GMM) can also yield station clusters. GMM is the soft version of k-means on the same features, so each station gets probabilities of belonging to each trip cluster. Every k-means round trip, weekend trip and weekday AM trip lands in the matching GMM group. They differ in the weekday afternoon trips. K-means splits them by distance (longer vs short-hop). GMM splits them by time (afternoon vs evening). Note in the charts in Appendix U2 that BIC

never levels off. Several binary features dominate, such as round trip and weekend, creating spikes that each extra Gaussian tries to fit.

Station Clusters

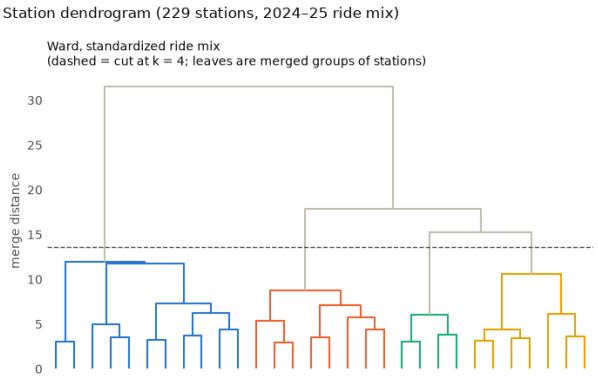
K-Means Station Clusters

We can create station clusters based on the mix of purposes expressed in the ride clusters. Based on separation, stability, and prediction-strength scores, we settled on k=4. While k=2 is the most robust split and has the best scores, k=4 offers the best balance of an informative yet stable division. See Appendix U1 for visuals of the metrics used to select k values for both k-means and GMM.

GMM Station Clusters

The soft-clustering counterpart of k-means on the same standardized ride mix: each station gets a probability of belonging to each component. This reflects that stations are proportional mixtures of trip purposes rather than discrete kinds. We used k = 4, the same as k-means, so we can compare the two typologies like for like. We named the components based on their profiles.

Figure U2. Station Dendrogram



Hierarchical for Comparison

Ward serves as a check on k-means, not a separate method. It minimizes the same quantity (spread within each cluster), but it does so by merging stations step by step rather than iterating from random starting centers.

This dendrogram shows the hierarchical approach. Appendix U2 provides details on each cluster. The first major split separates leisure stations from the rest, matching the k = 2 result. No long branch marks an obvious place to cut at k = 4. The cut falls among merges of similar height. The dendrogram reinforces the idea that the stations form a continuum and that k = 4 is a choice, not a discovery. It makes the same point DBSCAN makes, from a different angle.

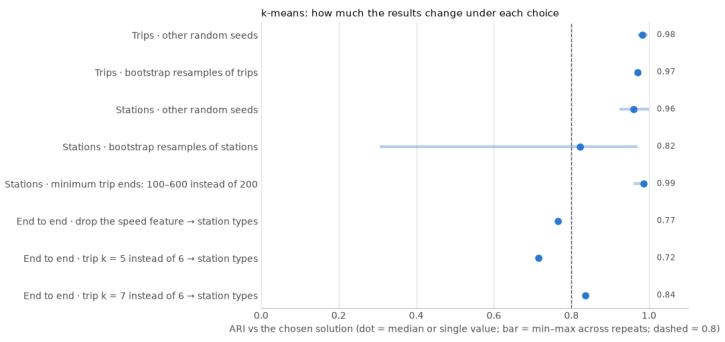
DBSCAN Didn't Prove Informative

DBSCAN asks whether trips and stations have natural gaps between groups: dense regions separated by sparse ones. This would imply real clusters rather than a continuum. We tried DBSCAN, but the purpose mix is a spectrum and therefore not a great fit for revealing clear station typology. DBSCAN is fine for detecting outliers among extreme leisure stations. There are no natural clusters to discover, so partitioning methods (k-means, GMM) are the right tools, and their boundaries should be read as segmentation.

Unsupervised Evaluation

Sensitivity Analysis

We measured whether the k-means results change when we change things that shouldn't matter, and how much they change when we change real modeling choices. The chart below illustrates this



analysis. Overall, the results are stable to randomness, sampling and thresholds.

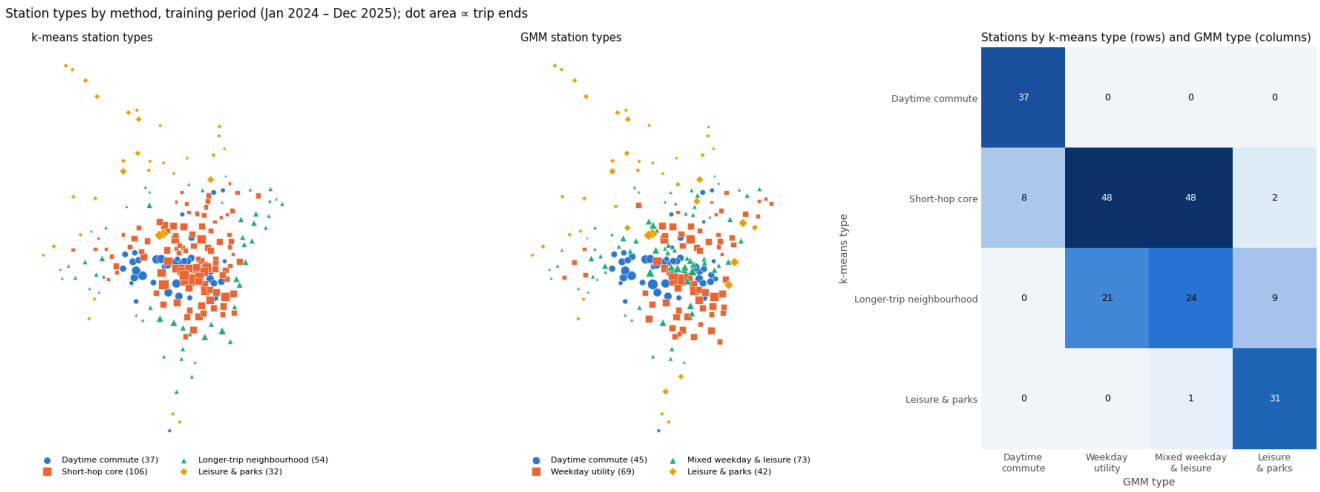
Figure U3. Sensitivity analysis of k-means

- *Random seed*: no effect. Different starting points provide nearly the same clusters for trips and stations (ARI 0.92–1.00).
- *Different data samples*: trips are very stable (0.97). Stations are less so (median 0.82). The broad station types hold up, but the exact boundaries depend on which stations are included.
- *Thresholds*: changing the minimum number of trip ends a station needs (100–600 instead of 200) makes almost no difference (0.96–0.99).
- *Number of station clusters (k)*: only the two-way split (leisure stations vs everything else) is clearly strongest. k = 4 is among the most stable of the finer splits, and its types are easy to interpret.

Clustering Results & Recommendation

The chart below reflects the results of both methods via a geographic and heatmap comparison. Both methods shared the names of two clusters, but differed on the other two. Appendix U3 compares metrics for multiple approaches for both ride and station clusters. The chart below shows the results of each clustering approach.

Figure U4: K-means and GMM by station location. Heat map comparing cluster counts for both methods.



Based on this analysis, k-means is the best option. While it leaves a judgment call for the specific k value, it is flexible enough to adapt to the two-phased approach and the time dominance of derived values. It offers the highest stability and competitive fit.

- Trips: k-means (k = 6). It is far more stable than Ward or GMM (0.98 vs 0.59 and 0.61), explains the most casual-share variance (ε² 0.079), and a depth-3 tree reproduces it 87% of the time.
- Stations: k-means (k=4) but use GMM membership probabilities to show how firmly each station belongs.

Table U5: Comparing metrics across attempted clustering methods

	clusters	coverage	fit (silhouette)	stability	held-out ε^2: casual	held-out ε^2: electric	interpretability	ARI vs reference
k-means (k = 6)	6	100%	0.242	0.98	0.079	0.021	87%	0.90
GMM (k = 6)	6	100%	0.219	0.61	0.051	0.018	88%	0.69
DBSCAN	5	96%	0.276	1.00	0.036	0.000	100%	0.28

	clusters	coverage	fit (silhouette)	stability	held-out ε^2: AM net flow	held-out ε^2: casual	interpretability	ARI vs reference
k-means pipeline (k = 4)	4	100%	0.275	0.89	0.244	0.313	86%	1.00
GMM pipeline (k = 4)	4	100%	0.183	0.80	0.305	0.378	79%	0.34
DBSCAN	1	98%	—	—	—	—	—	—

Discussion

We gained several key insights and even observed some surprises from performing this analysis.

Lesson Learned - Supervised

- *What surprised us*: The simple historical baseline was already quite strong. XGBoost performed best, but its improvement was modest, and it still missed sudden demand spikes. We learned that a more complex model doesn't automatically produce much better predictions.
- *Challenges and responses*: Missing data and limited station history made prediction difficult. We compared ways of handling missing values, used time-based validation, and examined individual failures to understand what the overall scores missed.

Lesson Learned - Unsupervised

- *Derived features*: A dataset with a few features, especially datetime values, can yield additional features that augment the analysis. See the Future Work section below for more details.
- *Behavior can be inferred*: Behavioral profiles can result from what you observe, not just from what you collect or what you are told by “subjects.”
- *Effectiveness over cleverness*: More model complexity does not yield better outcomes
- Judgement calls: There is still judgment needed to choose thresholds, cutoffs and starting points. requires optimizing across several performance variables, including predictive power, separation and interpretability.
- *Experiment and iterate*: Be prepare to accept some false starts and dead ends as part of the process.

Future Work

We kept the project focused on the core scope, given the complexity added by an enhanced data set and two-phased clustering approach. With more time for feature engineering, we could add additional layers of analysis in future iterations of the project.

- *Alternative segmentation*: Each station’s role in the network and the flow of inventory
- *Hourly station profiles*: Describe each station by its 96-value departure/arrival profile instead of the six purpose shares. This is the fine-grained time information that stage 2 sees only coarsely, and it could include direction.
- *Seasonality*: Does the clustering change or do stations change type between summer & winter
- *Mix + flow typology*: A five-type alternative that builds direction into the clustering.

- *Day regime clusters*: Use event, visitor, holiday and weather data to create a day typology. This could provide a 3rd phase to the cluster pipeline.
- *Station roster effects*: Do new stations add to ride potential in a neighborhood or do they cannibalize from other nearby stations? Is there a ramp-up period?
- *Improve event-related features:* Give the neural network more time to converge and evaluate the final model on a completely new period to see whether the improvement holds.

Ethical Considerations

Bias

We need to watch for bias baked into each station's location and what it implies about riders or what the time implies about the type of rider. Some stations use a virtual process, rather than being fully self-serve, and could leave an identification trail if not handled properly. Even the pass or bike type could introduce unintended bias.

Station locations may also introduce spatial and demographic bias, with certain populations and neighborhoods unable to access bike share stations. The timing of key events near stations, such as the collective arenas and stadium complex, could also imply certain characteristics of riders who are able to attend those events.

Aggregation & Privacy

The public dataset anonymizes individual rides. There is no identifiable information or rider IDs. It is unknown whether the source data that yields the public dataset contains this information. Credit card, passholder ID, or even app or phone identification data could be available and leaked if not handled properly. Although the time of day for a ride may likely indicate the “job” the ride is hired to do: commuting v. exploring. We will need to present trends across stations and rides, not individuals. Each new dataset included in any analysis will need the same review as the core public datasets provided by Indego.

Unintended Usage (Usage Creep)

Although we cannot assume informed consent for this anonymized data, we should ensure our analysis reflects the goal of understanding patterns and relationships between factors, not reducing service in bike share or public transportation networks.

Surveillance and Geographic Tracking

While the current dataset is anonymized, knowing a particular person was at a particular point at a particular time could still lead to unintended harms and consequences. This is especially true because mobile phone and app usage are critical to delivering the bike share experience.

Statement of Work

Xiaofan Tang	**Ian McCourt**	**Hal Newton**
Supervised models: spatiotemporal regression, evaluation, sensitivity analysis, and visualizations	Data processing & pipeline: Combine and clean the Indego data. Add weather, event, SEPTA, and DisPatch70 data	Project concept. Unsupervised models: trip and station clustering, evaluations and visualizations.

References

Chen, Z. (2019, November 12). *Space-time prediction of bike share demand*. https://zenithchen.github.io/content/Zirui_Chen_Bikeshare_Predict.html

Collins-Thompson, Kevyn. “Model Evaluation & Selection.” *SIADS 542: Supervised Learning*, Week 3, University of Michigan School of Information, Fall 2025. Lecture video (22 min), Canvas.

Collins-Thompson, Kevyn. “K-means Clustering.” *SIADS 543: Unsupervised Learning*, Week 2, University of Michigan School of Information, Winter 2026. Lecture video, Canvas.

City of Philadelphia. (2025, August 19). *What you need to know about the City’s response to the SEPTA service cuts* (Updated September 26, 2025). https://www.phila.gov/2025-08-19-city-response-septa-service-cuts/

City of Philadelphia, Office of Special Events. (2026, May 19). *Mayor Parker unveils transportation and access plan for FIFA Fan Festival Philadelphia*. https://www.phila.gov/2026-05-19-mayor-parker-unveils-transportation-and-access-plan-for-fifa-fan-festival-philadelphia/

Froehlich, J., Neumann, J., & Oliver, N. (2009). Sensing and predicting the pulse of the city through shared bicycling. In Proceedings of the 21st International Joint Conference on Artificial Intelligence (pp. 1420–1426). https://www.ijcai.org/Proceedings/09/Papers/238.pdf

hulnegy. (n.d.). *DisPatch70* [Data set]. Hugging Face. Retrieved October 16, 2026, from https://huggingface.co/datasets/hulnegy/DisPatch70

Indego. (n.d.). *Trip & station data*. Retrieved October 16, 2026, from https://www.rideindego.com/about/data/

Sandvig, Christian. “Module 3: Data Provenance, Aggregation, Trust.” *SIADS 503: Data Science Ethics*, Week 3, University of Michigan School of Information, Fall 2025. Lecture slides, Canvas.

Li, Y., Zheng, Y., Zhang, H., & Chen, L. (2015). Traffic prediction in a bike-sharing system. In Proceedings of the 23rd SIGSPATIAL International Conference on Advances in Geographic Information Systems (Article 33). Association for Computing Machinery. https://doi.org/10.1145/2820783.2820837

Appendices

Data Sources

Appendix D1: Data sources used in the project

Dataset (location)	Format	Key Variables	Records	Period Covered
Indego trip data (rideindego.com/about/data; Indego, n.d.)	10 quarterly ZIP files, one CSV each	trip_id, duration, start/end time, start/end station ID and coordinates, trip_route_category, passholder_type, bike_type, plan_duration	3,316,798 rows read; 3,316,664 kept; 338 start stations	2024 Q1–2026 Q2 (first trip 2024-01-01 00:04; last 2026-06-30 23:58)
Indego station table (same page)	CSV (indego-stations-2026-07-15.csv)	Station_ID, Station_Name, go-live date, Active/Inactive status	368 rows (311 Active, 57 Inactive), including Virtual Station 3000	Snapshot dated 2026-07-15
DisPatch70 situations, Philadelphia (situations/philadelphia/{year}.parquet)	Parquet, one file per year	30-minute city snapshots: temperature, apparent temperature, humidity, precipitation, rain, snowfall, wind, cloud cover, PM2.5, US AQI, weather code, holiday flag and name, day type, regime, typical_trips	joined to 100% of station-hours	2024–2026
DisPatch70 weather events (events/philadelphia/weather/{year}.parquet)	Parquet, one file per year	src_kind (rain, heavy rain, heat, cold, snow, wind, fog, air quality), start_local, end_local		2024–2026
DisPatch70 ESPN sports (events/philadelphia/sports_espn/{year}.parquet)	Parquet, one file per year	venue_name, league, start/end time, expected_attendance, status (STATUS_FINAL only)		2024–2026
DisPatch70 Indego station status (station_status/philadelphia/indego/{year}/{Mon}.parquet)	Parquet, one file per month	commit_at (UTC), station_id, bikes, stands (open docks)	25 monthly files: Apr and Aug–Dec 2024, all of 2025, Jan–Jul 2026	Apr 2024–Jul 2026 with gaps; covers 26.5% of panel station-hours
DisPatch70 dispatch (dispatch/philadelphia/{year}.parquet)	Parquet, one file per year	kind (move, service, …), from/to station name, t_from/t_to (epoch s), n_bikes	326 distinct station names	2024–2026

DisPatch70 station context (station_context/philadelphia/indego.parquet)	Parquet, single file	capacity, lat/lon, district, neighborhood, OSM point-of-interest counts, nearby transit, neighboring stations	Context available for 338 of 375 station-table rows	Static snapshot (date not documented)
Case-study calendars (City of Philadelphia, 2025; City of Philadelphia, Office of Special Events, 2026)	Hand-coded from official notices	SEPTA cut 2025-08-24 to 2025-09-13; fare increase from 2025-09-14; six FIFA kickoffs (five within the trip window)	1 service-cut window; 20 match hours on 5 dates	Aug 2025–Jul 2026

Supervised

Appendix S1: Errors for selected groups of test windows

Four-hour windows	Count	Model MAE	Baseline MAE
Zero departures	248,358	0.664	0.557
Positive departures	418,743	1.858	2.048
High demand at least 9 departures	61,558	4.442	4.931
Fewer than four weekly lags	11,599	2.138	2.352
Match overlap near stadium	245	10.332	10.861
Other June windows near stadium	4,774	1.266	1.307

Appendix S2: Why 89 test stations lack a saved station group

Verified grouping eligibility reason	Stations
No usable cleaned training trips	24
Fewer than 200 training trip ends	9
Fewer than 50 trip ends in at least one training quarter	56

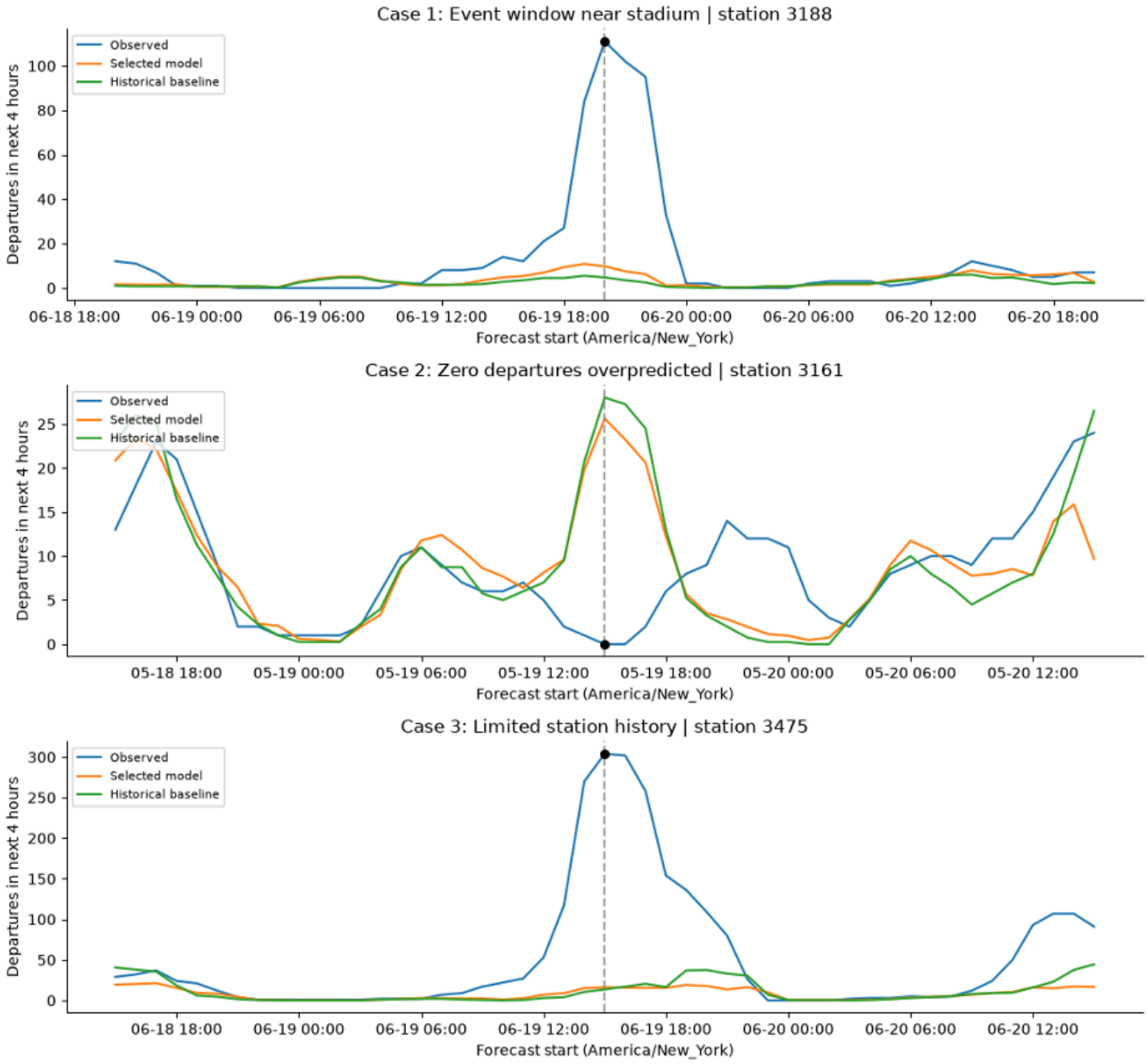
Appendix S3: Prediction errors across station groups

Station group	XGBoost MAE	Baseline MAE
Leisure & parks	0.9983	1.0330
Longer-trip neighbourhood	1.3208	1.3890
Daytime commute	1.7616	1.8422
Short-hop core	1.7786	1.9220
Ungrouped stations	1.0304	1.0521

Appendix S4: Details on failures

Station and forecast start	Observed	Model	Baseline	Absolute error
3188 Broad and Pattison BSL 19 June 2026 20:00	111	9.725	4.750	101.275
3161 30th Street Station East 19 May 2026 15:00	0	25.603	28.000	25.603
3475 Lemon Hill West 19 June 2026 15:00	304	16.251	13.500	287.749

Appendix S5: Failure traces and sources



Note: The vertical line marks the selected forecast start. Each point is an overlapping four-hour total, not an independent hourly count.

Unsupervised

Appendix U1 - Trip Clusters

Table U1.1: Details on derived features.

Raw column(s)	Derived clustering feature	What it captures
duration	log_duration	how long
start_lat, start_lon, end_lat, end_lon	log_dist: straight-line distance, km, log scale	how far
duration and the distance	speed_kmh: distance ÷ time, capped at 30	how direct (low = meandering)
trip_route_category	round_trip (0/1)	same start and end station
start_time	hour_sin, hour_cos: time of day as a point on a 24-hour circle	when in the day
start_time	weekend (0/1)	weekday or weekend

Figure U1.1: K-means inertia and silhouette scores by cluster

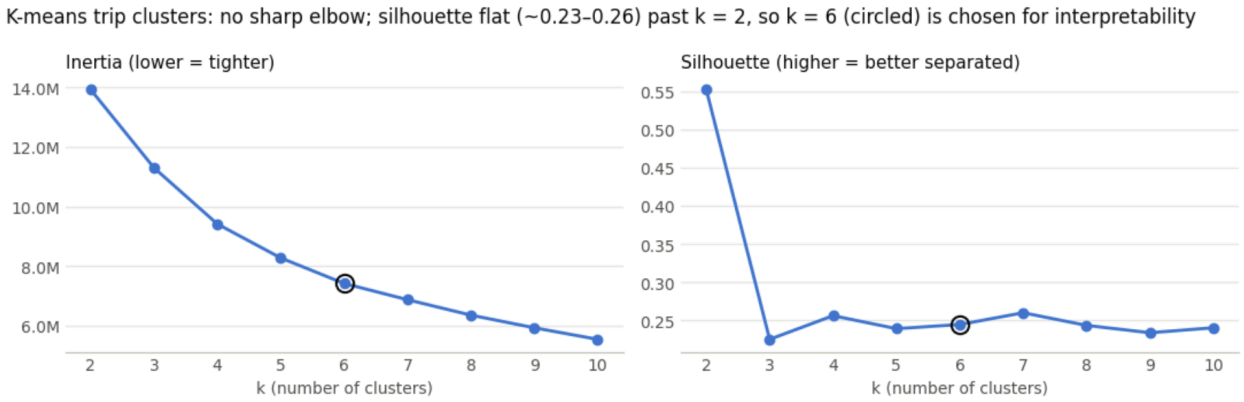


Figure U1.2: GMM BIC and AIC scores by components.

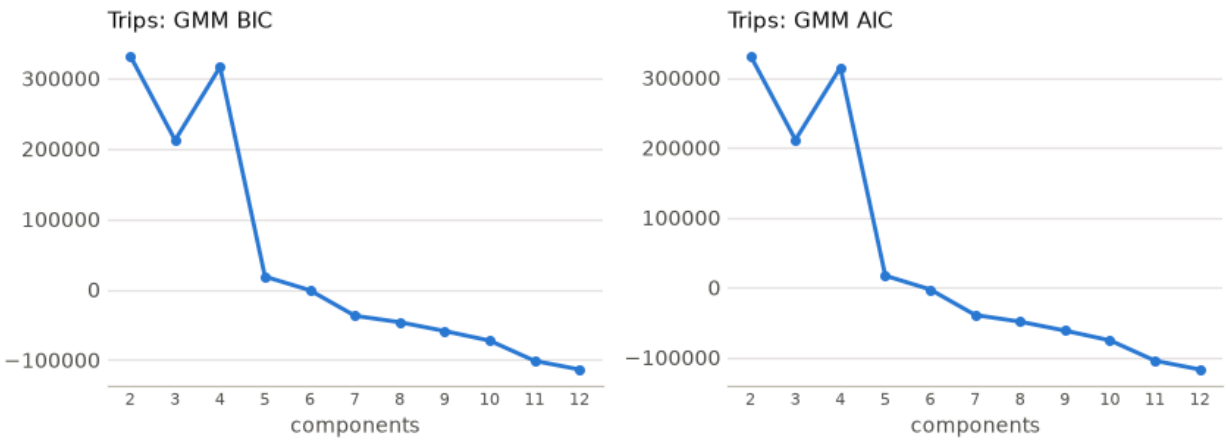
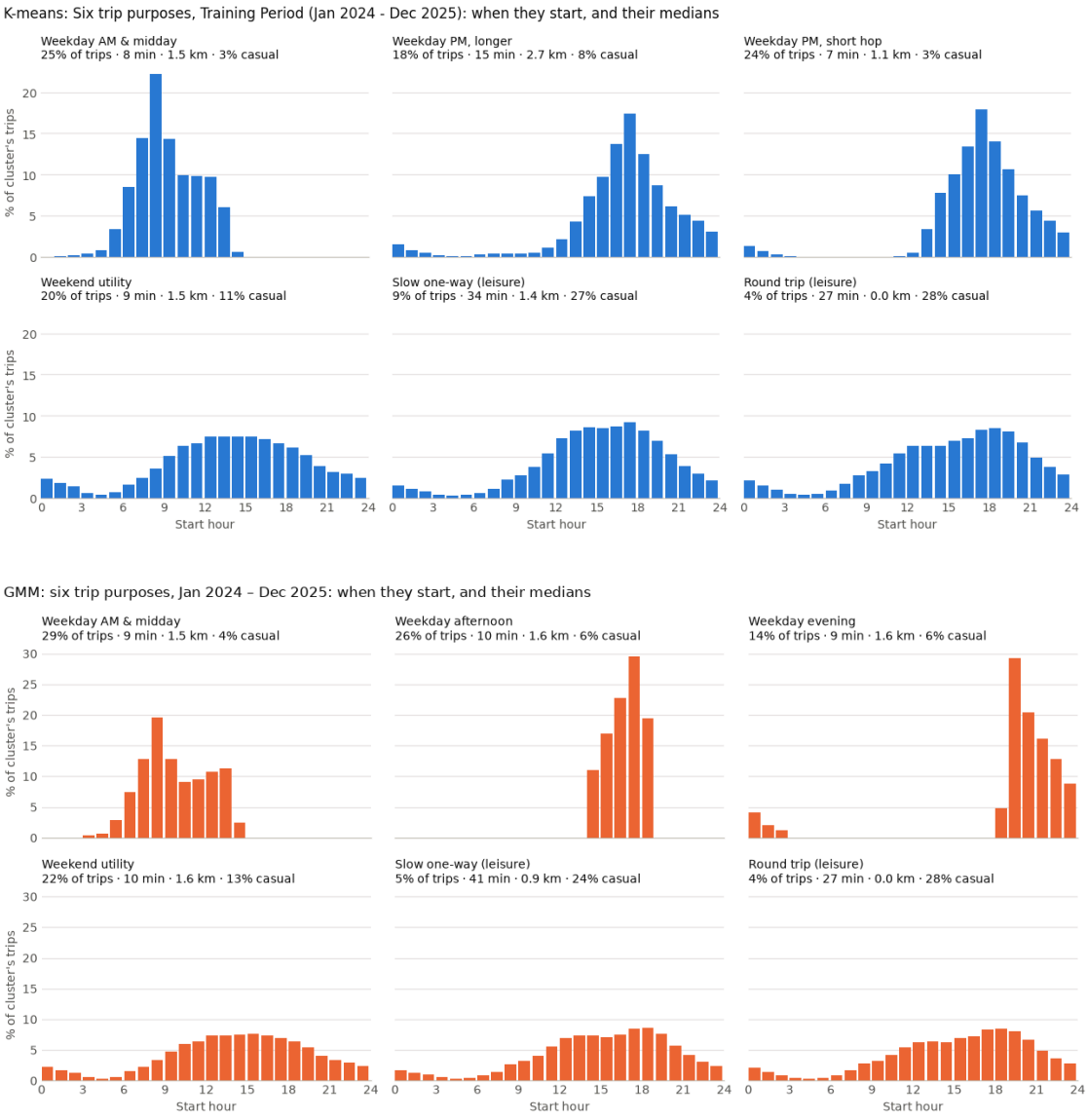
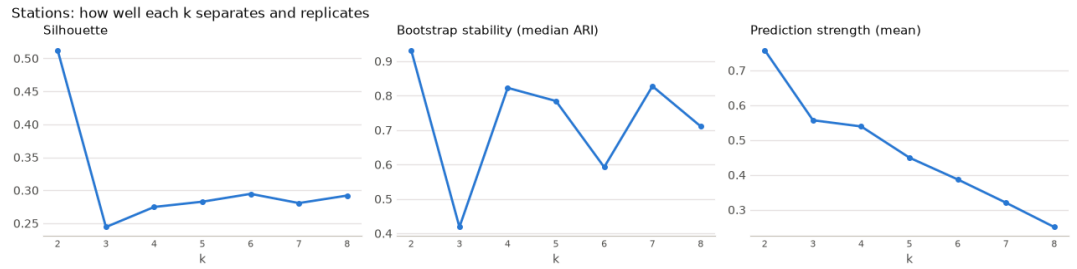


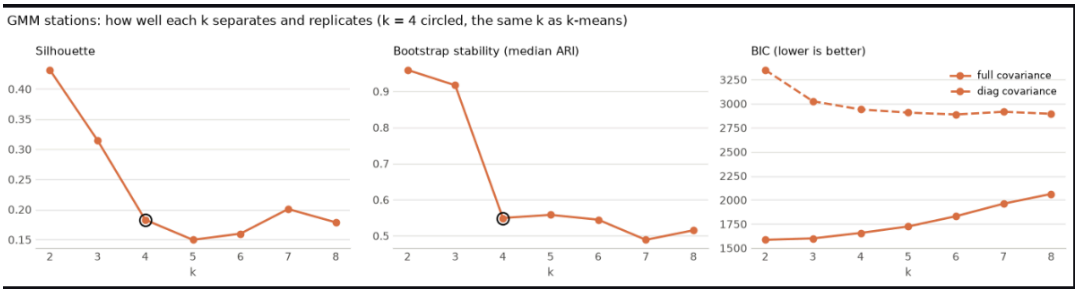
Figure U1.3: Comparing models by cluster profiles by hour



Appendix U2 - Station Clusters

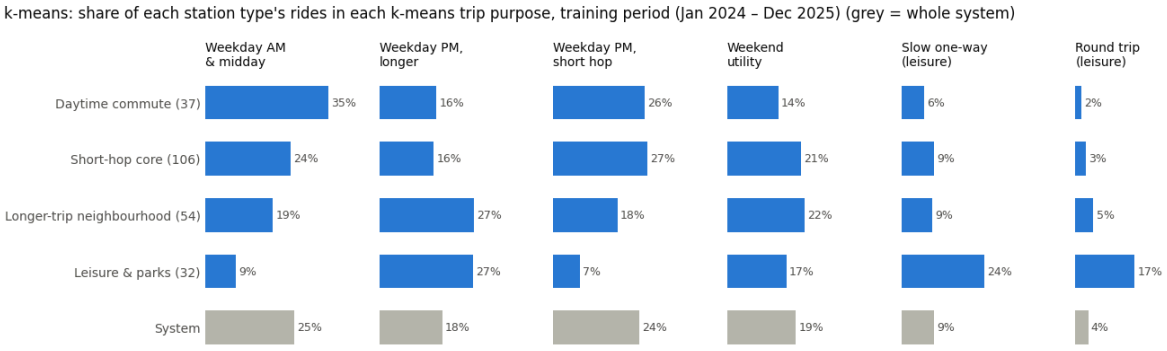
Figure U2.1: Comparing models by silhouette, bootstrap stability and prediction strength





Appendix U3: Comparing clusters by model

Figure U3.1: Comparing models by comparing distribution of station clusters by trip purpose clusters



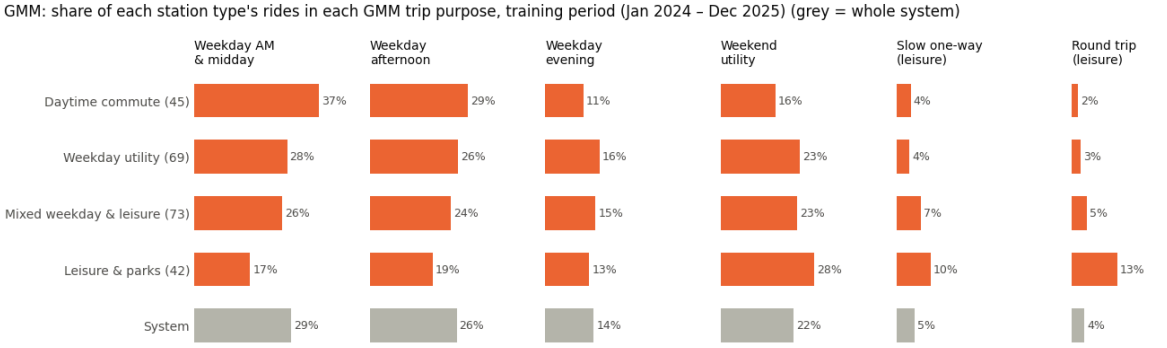


Figure U3.2: Trip purposes by method

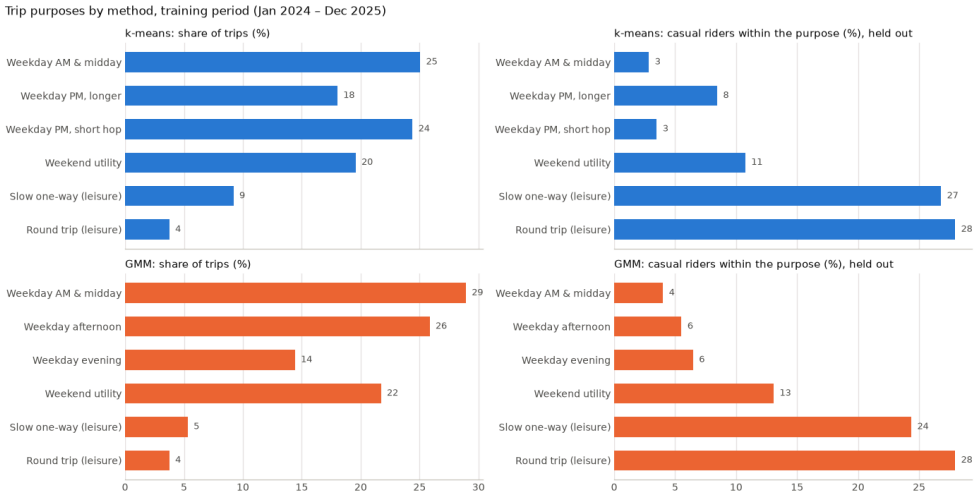


Figure U3.3: Comparing models with biplots

